UNISENAI FLORIANÓPOLIS

PROJETO APLICADO I

AUTOMAÇÃO DA EMISSÃO DE CERTIFICADOS DE CALIBRAÇÃO

AV03 — MVP Parte 1: Programação Orientada a Objetos e Persistência de Dados

Cliente: Schulz S.A. | Orientador: Prof. Hugo

Equipe: Gabriel Oliveira Born, Gustavo Felicio de Lacerda, Iduilton Junior de Souza Santos, Mateus Foltran Tomaz, Mateus Paterno

Florianópolis/SC — 03 de novembro de 2026

SUMÁRIO

SUMÁRIO 2
1. INTRODUÇÃO E ESCOPO DA AV03 3
2. ARQUITETURA DA SOLUÇÃO 3
2.1. Visão geral em camadas 3
2.2. Diagrama UML textual das classes 4
3. APLICAÇÃO DOS PILARES DA PROGRAMAÇÃO ORIENTADA A OBJETOS 6
3.1. Abstração 6
3.2. Encapsulamento 7
3.3. Herança 7
3.4. Polimorfismo 7
4. MODELO DE DOMÍNIO E REGRAS DE NEGÓCIO IMPLEMENTADAS 8
4.1. Mapeamento objeto-relacional 8
4.2. Regras de negócio validadas no código 8
4.3. Lógica de classificação do parecer 8
5. TRATAMENTO DE ERROS E PERSISTÊNCIA 9
5.1. Tabela de tratamento de erros SQL 9
5.2. Padrão Singleton e controle transacional 9
5.3. Consultas parametrizadas e prevenção a SQL Injection 9
6. DEMONSTRAÇÃO DE EXECUÇÃO 9
7. JUSTIFICATIVA TÉCNICA: MANUTENÇÃO DO MYSQL 10
8. CONSIDERAÇÕES FINAIS E PRÓXIMOS PASSOS 11
APÊNDICE A — CÓDIGO-FONTE: 04_classes.py 12
APÊNDICE B — CÓDIGO-FONTE: 05_main.py 19
APÊNDICE C — INSTRUÇÕES DE EXECUÇÃO 24

1. INTRODUÇÃO E ESCOPO DA AV03

A presente entrega corresponde à terceira etapa (AV03) do Projeto Aplicado I, disciplina integrante do curso de graduação oferecido pelo UniSENAI Florianópolis em parceria com a empresa Schulz S.A. Esta terceira etapa dá continuidade ao ciclo de desenvolvimento iniciado na AV01, que levantou e consolidou os requisitos funcionais e não funcionais do sistema de automação da emissão de certificados de calibração, e na AV02, que definiu o modelo de dados relacional — representado por nove tabelas inter-relacionadas — bem como as regras de negócio que governam a operação metrológica do laboratório.

O objetivo central da AV03 é implementar a camada de software orientada a objetos que consome o banco relacional já modelado nas etapas anteriores. Para tanto, a equipe desenvolveu uma aplicação em Python que aplica, de forma prática e verificável, os quatro pilares da Programação Orientada a Objetos (POO) — abstração, encapsulamento, herança e polimorfismo — e adota boas práticas de persistência de dados, como o padrão DAO (Data Access Object), consultas parametrizadas, controle transacional explícito e tratamento estruturado de exceções, sempre tomando como base as regras de negócio (RN01 a RN06) homologadas junto à Schulz S.A.

O escopo efetivamente entregue nesta AV03 compreende dois arquivos-fonte em Python — 04_classes.py, com as entidades do domínio, a conexão Singleton e as classes DAO, e 05_main.py, com a classe orquestradora SistemaCalibracao e a demonstração do fluxo de execução —, a documentação técnica em Markdown (06_mvp_parte1.md) e o presente relatório consolidado, que registra as decisões arquiteturais, a aplicação dos pilares da POO, o tratamento de erros e os resultados da demonstração de execução.

Decisão arquitetural destacada: a equipe deliberadamente optou por manter o MySQL 8.0 como SGBD, em vez de migrar para o SQLite sugerido pelo roteiro da AV03. A justificativa técnica completa para essa escolha é apresentada na Seção 7 deste relatório e fundamenta-se na continuidade do projeto entre entregas, na preservação da integridade referencial já validada, na aderência ao ambiente real da Schulz S.A. e na aproximação com o cenário profissional de gerenciamento de bancos de dados. As três entidades simplificadas propostas pelo roteiro foram integralmente contempladas pelas classes Tecnico, Equipamento e Medicao, apenas mapeadas para as tabelas equivalentes do modelo completo.

2. ARQUITETURA DA SOLUÇÃO

2.1. Visão geral em camadas

A solução foi estruturada segundo uma arquitetura em camadas que separa responsabilidades e reduz o acoplamento entre a interface, o domínio e o acesso a dados. A Camada de Apresentação é responsável pela orquestração do fluxo e pela interação com o usuário, sendo materializada no arquivo 05_main.py por meio da classe SistemaCalibracao. Essa classe delega as operações de negócio às entidades e não conhece os detalhes de persistência, comunicando-se exclusivamente por intermédio das classes DAO.

A Camada de Modelo/Domínio concentra as entidades do domínio metrológico — Tecnico, Equipamento e Medicao —, que encapsulam os atributos relevantes a cada conceito e as regras de negócio associadas, como validação de e-mail, cálculo de erro por soma algébrica, verificação de calibração vencida e classificação do parecer. Essas classes

independem de qualquer biblioteca de banco de dados e permanecem validáveis por testes unitários isolados.

A Camada de Persistência agrupa as classes TecnicoDAO, EquipamentoDAO e MedicaoDAO, além da classe Conexao, que implementa o padrão Singleton para o gerenciamento da conexão com o MySQL. O padrão DAO (Data Access Object) isola todo o SQL e as operações de leitura/escrita da lógica de domínio: cada classe DAO concentra as instruções INSERT, SELECT, UPDATE e DELETE referentes à sua entidade e converte as linhas vindas do banco em objetos do domínio (e vice-versa), de modo que as classes de domínio nunca são contaminadas por detalhes de persistência e o banco pode ser trocado com impacto mínimo no restante da aplicação.

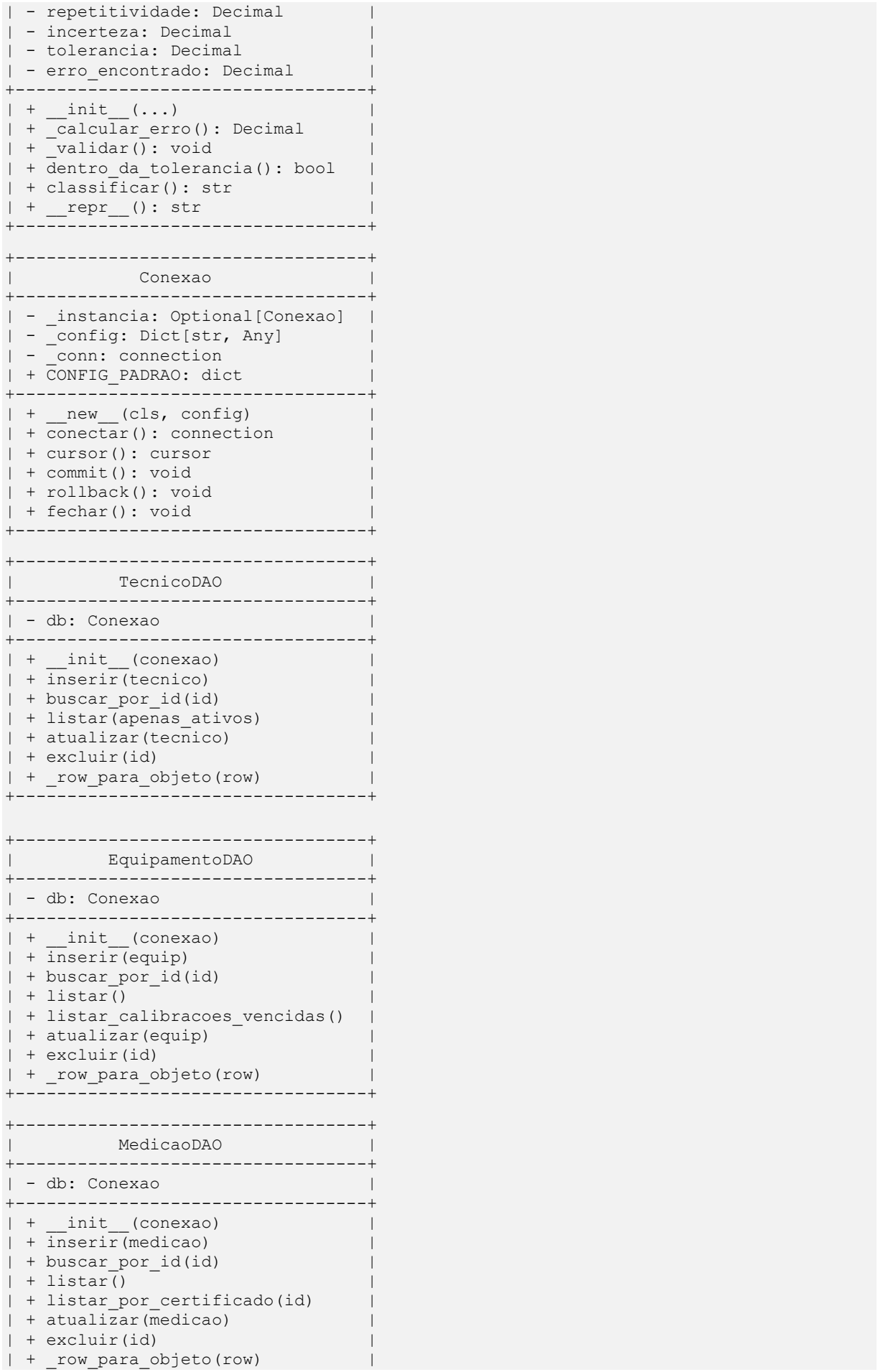
2.2. Diagrama UML textual das classes

O diagrama a seguir representa, em notação textual ASCII, as principais classes da solução, seus atributos e métodos, bem como as relações de associação e dependência entre elas. As entidades do domínio são independentes; as classes DAO possuem associação com suas respectivas entidades e com a conexão; e a classe SistemaCalibracao orquestra os DAOs.

```
+----------------------------------+
|            Tecnico               |
+----------------------------------+
| - id_usuario: int                |
| - nome: str                      |
| - email: str                     |
| - senha_hash: str                |
| - papel: str                     |
| - registro_profissional: str     |
| - ativo: bool                    |
+----------------------------------+
| + __init__(...)                  |
| + _validar(): void               |
| + __repr__(): str                |
| + PAPEIS_VALIDOS: set            |
+----------------------------------+

+----------------------------------+
|           Equipamento            |
+----------------------------------+
| - id_instrumento: int            |
| - tag: str                       |
| - descricao: str                 |
| - tipo: str                      |
| - fabricante: str                |
| - modelo: str                    |
| - numero_serie: str              |
| - resolucao_mm: Decimal          |
| - data_calibracao_prevista: date |
| - id_cliente: int                |
+----------------------------------+
| + __init__(...)                  |
| + _validar(): void               |
| + calibracao_vencida(): bool     |
| + dias_para_vencer(): int        |
| + __repr__(): str                |
+----------------------------------+

+----------------------------------+
|            Medicao               |
+----------------------------------+
| - id_medicao: int                |
| - ponto_medicao: str             |
| - valor_nominal: Decimal         |
| - valor_medido: Decimal          |
```


- repetitividade: Decimal
- incerteza: Decimal
- tolerancia: Decimal
- erro_encontrado: Decimal
+ __init__(...)
+ _calcular_erro(): Decimal
+ _validar(): void
+ dentro_da_tolerancia(): bool
+ classificar(): str
+ __repr__(): str
Conexao
- _instancia: Optional[Conexao]
- _config: Dict[str, Any]
- _conn: connection
+ CONFIG_PADRAO: dict
+ __new__(cls, config)
+ conectar(): connection
+ cursor(): cursor
+ commit(): void
+ rollback(): void
+ fechar(): void
TecnicoDAO
- db: Conexao
+ __init__(conexao)
+ inserir(tecnico)
+ buscar_por_id(id)
+ listar(apenas_ativos)
+ atualizar(tecnico)
+ excluir(id)
+ _row_para_objeto(row)
EquipamentoDAO
- db: Conexao
+ __init__(conexao)
+ inserir(equip)
+ buscar_por_id(id)
+ listar()
+ listar_calibracoes_vencidas()
+ atualizar(equip)
+ excluir(id)
+ _row_para_objeto(row)
MedicaoDAO
- db: Conexao
+ __init__(conexao)
+ inserir(medicao)
+ buscar_por_id(id)
+ listar()
+ listar_por_certificado(id)
+ atualizar(medicao)
+ excluir(id)
+ _row_para_objeto(row)

```
+----------------------------------+


+----------------------------------+
|       SistemaCalibracao          |
+----------------------------------+
| - db: Conexao                    |
| - tecnicos: TecnicoDAO           |
| - equipamentos: EquipamentoDAO   |
| - medicoes: MedicaoDAO           |
+----------------------------------+
| + __init__()                     |
| + testar_conexao(): bool         |
| + demonstrar_crud(): void        |
| + demonstrar_validacoes(): void  |
| + relatorio_certificados()       |
| + relatorio_pendentes()          |
| + relatorio_rastreabilidade()    |
| + relatorio_alertas()            |
| + executar(): void               |
| + encerrar(): void               |
+----------------------------------+

Relações:

  TecnicoDAO ..> Tecnico : gerencia
  EquipamentoDAO ..> Equipamento : gerencia
  MedicaoDAO ..> Medicao : gerencia
  TecnicoDAO ..> Conexao : usa
  EquipamentoDAO ..> Conexao : usa
  MedicaoDAO ..> Conexao : usa
  SistemaCalibracao --> TecnicoDAO : injeta
  SistemaCalibracao --> EquipamentoDAO : injeta
  SistemaCalibracao --> MedicaoDAO : injeta
  SistemaCalibracao --> Conexao : orquestra
  Conexao ..> MySQL 8.0 : conector
  ErroValidacao ..> Exception : herda
  ErroPersistencia ..> Exception : herda
```

3. APLICAÇÃO DOS PILARES DA PROGRAMAÇÃO ORIENTADA A OBJETOS

3.1. Abstração

A abstração consiste em representar apenas os aspectos essenciais de um conceito do mundo real, omitindo detalhes de implementação. As classes Tecnico, Equipamento e Medicao abstraem as entidades do domínio metrológico, expondo apenas os atributos e comportamentos relevantes ao negócio e escondendo os detalhes de persistência. Um exemplo é a classe Medicao, cujo construtor recebe apenas os valores metrológicos (nominal, medido, repetitividade, incerteza e tolerância) e oculta internamente como o erro é calculado; a classe de domínio não conhece o banco de dados. O trecho a seguir ilustra o construtor da classe Medicao:

```
class Medicao:
    """Ponto de medicao dimensional. Mapeia a tabela 'medicao'."""

    def __init__(self, ponto_medicao, valor_nominal, valor_medido,
                 repetitividade, incerteza, tolerancia=Decimal("0.0100"),
                 id_medicao=None):
        self.id_medicao = id_medicao
        self.ponto_medicao = ponto_medicao
        self.valor_nominal = Decimal(str(valor_nominal))
        self.valor_medido = Decimal(str(valor_medido))
        self.repetitividade = Decimal(str(repetitividade))
        self.incerteza = Decimal(str(incerteza))
        self.tolerancia = Decimal(str(tolerancia))
        self.erro_encontrado = self._calcular_erro()
        self._validar()
```

3.2. Encapsulamento

O encapsulamento protege o estado interno de um objeto, controlando o acesso aos seus atributos e garantindo a invariância do sistema. Na solução, os atributos são privados/protegidos (convenção com sublinhado), a validação é centralizada no construtor por meio do método _validar(), e as violações são sinalizadas por meio das exceções personalizadas ErroValidacao e ErroPersistencia. Dessa forma, um objeto inválido jamais é criado ou persistido. O método _validar() da classe Tecnico demonstra esse comportamento:

```
    def _validar(self):
        if not self.nome or len(self.nome.strip()) < 3:
            raise ErroValidacao("Nome do tecnico deve ter ao menos 3 caracteres.")
        padrao_email = r"^[\w\.\-]+@[\w\.\-]+\.\w{2,}$"
        if not re.match(padrao_email, self.email or ""):
            raise ErroValidacao(f"E-mail invalido: {self.email}")
        if self.papel not in self.PAPEIS_VALIDOS:
            raise ErroValidacao(f"Papel invalido: {self.papel}")
        if self.papel == "SIGNATARIO" and not self.registro_profissional:
            raise ErroValidacao("Signatario exige registro profissional (RN06).")
```

3.3. Herança

A solução utiliza herança de forma implícita: como toda classe em Python, as entidades Tecnico, Equipamento e Medicao herdam, direta ou indiretamente, da classe base object, o que lhes confere os métodos padrão da linguagem. As exceções personalizadas ErroValidacao e ErroPersistencia, por sua vez, são subclasses explícitas de Exception, herdando sua interface de rastreabilidade e propagação de erros. A arquitetura, porém, foi deliberadamente construída com base em composição — as classes DAO recebem a instância de Conexao em seus construtores — em detrimento de uma hierarquia de herança profunda. Essa escolha foi tomada porque as entidades do domínio não compartilham uma árvore de comportamento comum relevante ao negócio, sendo entidades heterogêneas; a composição oferece maior flexibilidade, facilidade de teste (injeção de conexão) e menor acoplamento do que uma cadeia de herança artificial, além de evitar o acoplamento frágil típico de hierarquias profundas.

As classes Tecnico, Equipamento e Medicao não formam uma hierarquia entre si — cada uma representa um conceito distinto do domínio metrológico —, e os DAOs não herdam de uma classe base comum, priorizando composição em vez de duplicação estrutural. Já a herança tem papel central nas exceções, unificando o tratamento de erros de validação (ErroValidacao) e de persistência (ErroPersistencia) sob a hierarquia padrão de Exception.

3.4. Polimorfismo

O polimorfismo é aplicado de duas formas na solução. Na primeira, por meio da sobrescrita do método __repr__ em cada entidade, que redefine a representação textual objeto-a-objeto, permitindo que cada classe forneça sua própria forma de auto-descrição — o mesmo nome de método se comporta de modo distinto conforme a classe. Na segunda, por meio do duck typing nos métodos de listagem dos DAOs: todos os métodos listar(), buscar_por_id() e _row_para_objeto() seguem o mesmo contrato de assinatura e retornam objetos de suas respectivas entidades, de modo que o chamador trabalha com a interface comum sem conhecer o tipo concreto. O trecho a seguir ilustra a sobrescrita de __repr__ na classe Equipamento:

```
    def __repr__(self):
        return (f"<Equipamento id={self.id_instrumento} tag='{self.tag}' tipo='{self.tipo}'>")
```

4. MODELO DE DOMÍNIO E REGRAS DE NEGÓCIO IMPLEMENTADAS

4.1. Mapeamento objeto-relacional

O quadro a seguir apresenta o mapeamento entre as classes Python do domínio e as tabelas do banco MySQL 8.0, conforme o modelo relacional definido na AV02:

Classe Python	Tabela MySQL	Observação
Tecnico	usuario	Colaborador do laboratório (metrologista, signatário ou qualidade).
Equipamento	instrumento	Instrumento de medição (micrômetro, paquímetro, etc.).
Medicao	medicao	Ponto de medição dimensional com erro, repetitividade e incerteza.

4.2. Regras de negócio validadas no código

As regras de negócio homologadas na AV01 foram implementadas diretamente nas classes de domínio conforme descrito abaixo:

Regra (AV01)	Descrição	Onde é validada	Comportamento em caso de violação
RN03	Calibração vencida quando a data prevista é anterior à data atual.	Métodos calibracao_vencida() e listar_calibracoes_vencidas() (Equipamento e EquipamentoDAO).	Equipamento vencido é listado no relatório de alertas (Relatório 7).
RN05	Erro de medição calculado por soma algébrica (medido − nominal).	Método _calcular_erro() da classe Medicao.	Erro calculado automaticamente; valores inconsistentes são sinalizados pela validação.
RN06	Signatário exige registro profissional preenchido.	Método _validar() da classe Tecnico.	ErroValidacao: "Signatario exige registro profissional (RN06)."
—	Formato de e-mail validado por expressão regular.	Método _validar() da classe Tecnico.	ErroValidacao: "E-mail invalido: <valor>".
—	Tolerância dimensional avaliada por módulo do erro.	Métodos dentro_da_tolerancia() e classificar() da classe Medicao.	Parecer classificado como APROVADO, APROVADO_COM_RESTRICAO ou REJEITADO.
—	Papel do técnico deve pertencer ao conjunto de papéis válidos.	Método _validar() da classe Tecnico.	ErroValidacao: "Papel invalido: <valor>".

4.3. Lógica de classificação do parecer

A classe Medicao centraliza a classificação do parecer metrológico de cada ponto de medição com base no módulo do erro encontrado e na tolerância dimensional. O parecer é classificado da seguinte forma: (i) APROVADO quando o módulo do erro é menor ou igual a 80% da tolerância — o ponto atende folgadamente à especificação; (ii) APROVADO_COM_RESTRICAO quando o módulo do erro situa-se entre 80% e 100% da tolerância — o ponto está conforme, porém próximo do limite, demandando monitoramento; e (iii) REJEITADO quando o módulo do erro ultrapassa a tolerância — o ponto viola a especificação e reprova a medição. Essa lógica está implementada nos métodos

dentro_da_tolerancia() e classificar(), garantindo julgamento uniforme e auditável em todos os certificados.

5. TRATAMENTO DE ERROS E PERSISTÊNCIA

5.1. Tabela de tratamento de erros SQL

As classes DAO capturam as exceções do driver mysql-connector-python e as convertem em exceções de domínio com mensagens amigáveis ao usuário, preservando a causa original. O quadro a seguir resume o tratamento dos principais códigos de erro do MySQL:

Código MySQL	Significado	Tratamento na aplicação
1062	Entrada duplicada (chave única/primária violada).	ErroPersistencia com mensagem amigável, ex.: "E-mail ja cadastrado" (Tecnico) ou "Tag ja cadastrada" (Equipamento).
1451	Violação de FK em DELETE (registro referenciado).	Bloqueio com mensagem explicativa, ex.: "Entidade possui certificados vinculados e não pode ser excluída".
1452	FK inexistente em INSERT (cliente inválido).	ErroPersistencia: "Cliente inexistente (id=...)" ao inserir equipamento.
—	Falha de conexão genérica com o servidor.	ErroPersistencia em Conexao.conectar(): "Falha ao conectar ao MySQL: <detalhe>".

5.2. Padrão Singleton e controle transacional

A classe Conexao implementa o padrão Singleton: por meio da sobrescrita de __new__, garante-se que exista apenas uma única instância da conexão em todo o processo, à qual todos os DAOs se referenciam. Essa decisão evita a abertura de múltiplas conexões redundantes, reduz o consumo de recursos do servidor e assegura consistência no estado transacional da aplicação. O controle transacional é feito de forma explícita: toda operação de escrita (INSERT, UPDATE, DELETE) abre uma transação (o autocommit é mantido desabilitado), executa a instrução e, em caso de sucesso, chama commit(); em caso de falha capturada, a operação aciona rollback() antes de relançar a exceção de domínio, garantindo que o banco nunca permaneça em estado intermediário parcialmente atualizado.

5.3. Consultas parametrizadas e prevenção a SQL Injection

Todas as instruções SQL da solução são construídas com consultas parametrizadas, em que os valores são passados como placeholders %s segundo a sintaxe do mysql-connector-python e fornecidos separadamente na execução por meio de tuplas de parâmetros. Dessa forma, o driver trata os valores como dados e não como parte da instrução, impossibilitando a injeção de código SQL malicioso. Essa prática elimina a montagem de strings SQL por concatenação e protege a aplicação contra um dos vetores de ataque mais comuns em sistemas conectados a bancos de dados, reforçando a segurança e a confiabilidade da persistência.

6. DEMONSTRAÇÃO DE EXECUÇÃO

O arquivo 05_main.py, por meio da classe SistemaCalibracao, executa sete blocos de demonstração em sequência, cobrindo a conexão, o CRUD, as validações de regras de negócio e os relatórios. O fluxo completo é o seguinte:

1. **Bloco 1 —** Teste de conexão: conecta ao MySQL 8.0, consulta VERSION(), DATABASE() e NOW(), e valida a disponibilidade do banco bd_certificados_schulz.
2. **Bloco 2 —** Demonstração CRUD: INSERT de técnico (2.1), INSERT de equipamento (2.2), INSERT de medição com validação de tolerância (2.3), SELECT de listagem de técnicos ativos (2.4), UPDATE de dados (2.5) e DELETE de medição (2.6).
3. **Bloco 3 —** Validação de regras de negócio com captura de exceções: RN06 (signatário sem registro), e-mail inválido, RN05 (cálculo do erro por soma algébrica) e tolerância fora do limite.
4. **Bloco 4 —** Relatório de certificados com JOIN e contagem de medições: junta certificado, instrumento, cliente e a contagem de medições por certificado.
5. **Bloco 5 —** Relatório de certificados pendentes e aprovados: filtra certificados nos status PENDENTE_APROVACAO e APROVADO, com metrologista e signatário.
6. **Bloco 6 —** Relatório de rastreabilidade medição-padrão: relaciona medições aos padrões de calibração utilizados e seus laudos de validação.
7. **Bloco 7 —** Alertas de calibrações vencidas (RN03): lista equipamentos cuja data de calibração prevista já expirou.

A seguir, apresenta-se um exemplo de saída esperada no terminal, ilustrando o cabeçalho do sistema, o teste de conexão e um trecho da tabela de listagem de técnicos ativos:

```
############################################################
#                                                          #
#   SISTEMA DE CERTIFICADOS DE CALIBRACAO - SCHULZ S.A.    #
#   AV03 - MVP Parte 1 | POO + Persistencia MySQL          #
#                                                          #
############################################################

======================================================================
  1. TESTE DE CONEXAO COM O BANCO DE DADOS
======================================================================
  Servidor MySQL        8.0.36
  Banco de dados        bd_certificados_schulz
  Data/hora servidor    2026-11-03 10:15:22

  [OK] Conexao estabelecida com sucesso.

------------------------------------------------------------------------------
  2.4 SELECT - Listagem de tecnicos ativos
------------------------------------------------------------------------------
  Total encontrado: 3

  ID    NOME                            PAPEL          REGISTRO
  ----- ------------------------------- -------------- ----------------
  1     Ana Paula Ferreira              METROLOGISTA   TEC-MET-9901/SC
  2     Carlos Eduardo Mello            METROLOGISTA   TEC-MET-8845/SC
  8     Beatriz Souza                   SIGNATARIO     TEC-SIG-1234/SC
```

7. JUSTIFICATIVA TÉCNICA: MANUTENÇÃO DO MYSQL

O roteiro da AV03 sugere a utilização do SQLite como SGBD, com um modelo simplificado composto por três tabelas (tecnicos, equipamentos e medicoes). A equipe, no entanto, decidiu manter o MySQL 8.0 e o modelo completo de nove tabelas definido na AV02. Essa decisão se justifica por quatro razões principais. A primeira é a continuidade e evolução do projeto entre entregas: manter o mesmo SGBD e o mesmo modelo de dados permite reaproveitar integralmente o trabalho da AV02 e evoluir a solução de forma incremental, sem retrabalho de migração de schema ou de scripts de carga.

A segunda razão é a preservação da integridade referencial e das constraints já validadas. O modelo de nove tabelas, com suas chaves primárias, chaves estrangeiras e regras de unicidade, foi homologado e testado nas entregas anteriores; migrar para um SGBD mais

simples implicaria abandonar ou reimplementar essas garantias. A terceira razão é a aderência ao ambiente real da Schulz S.A., que opera com SGBD servidor em produção — desenvolver e validar a solução sobre o mesmo tipo de infraestrutura aproxima a aplicação das condições reais de implantação e evita surpresas na transição. A quarta e última razão é a demonstração de competência técnica no uso de driver de conexão externa (mysql-connector-python), cenário mais próximo do perfil profissional esperado para desenvolvedores de integração de sistemas.

Cabe ainda esclarecer que o modelo simplificado de três entidades proposto pelo roteiro (tecnicos, equipamentos, medicoes) foi integralmente contemplado pela solução: as classes Tecnico, Equipamento e Medicao implementam exatamente esses três conceitos, apenas mapeadas para as tabelas equivalentes do modelo completo — usuario, instrumento e medicao, respectivamente. Dessa forma, a solução atende à essência pedagógica do roteiro (demonstrar POO e persistência sobre três entidades centrais) sem abrir mão do rigor arquitetural e da integridade do modelo real da AV02.

8. CONSIDERAÇÕES FINAIS E PRÓXIMOS PASSOS

A AV03 entregou uma camada de software orientada a objetos madura e funcional, capaz de consumir o banco relacional definido na AV02 e de operar as regras de negócio homologadas na AV01. A solução comprova o domínio técnico da equipe nos quatro pilares da POO, no padrão DAO, na persistência transacional, no tratamento estruturado de erros e na prevenção a SQL Injection, além de exercitar a integração com o MySQL 8.0 em um cenário próximo do ambiente profissional da Schulz S.A. Como próximos passos do projeto, a equipe prevê: (i) a implementação da camada de interface (MVP Parte 2), que dará uma frente de interação amigável ao usuário; (ii) a geração automatizada do PDF do certificado de calibração; (iii) a integração com a leitura de arquivos de MMC (máquinas de medição por coordenadas); e (iv) a implementação da autenticação de usuários, completando o ciclo de automação da emissão de certificados.

APÊNDICE A — CÓDIGO-FONTE: 04_classes.py

```
"""
PROJETO APLICADO I - UniSENAI Florianopolis
AV03 - MVP Parte 1: Programacao Orientada a Objetos e Persistencia
Arquivo: 04_classes.py
SGBD: MySQL 8.0 | Driver: mysql-connector-python
"""

import re
from datetime import date
from decimal import Decimal
from typing import Optional, List, Dict, Any

import mysql.connector
from mysql.connector import Error as MySQLError



class ErroValidacao(Exception):
    """Lancada quando um atributo de dominio viola uma regra de negocio."""
    pass



class ErroPersistencia(Exception):
    """Lancada quando uma operacao de banco de dados falha."""
    pass



class Conexao:
    """Gerencia a conexao com o MySQL (padrao Singleton)."""

    _instancia: Optional["Conexao"] = None

    CONFIG_PADRAO = {
        "host": "localhost",
        "port": 3306,
        "user": "root",
        "password": "",
        "database": "bd_certificados_schulz",
        "charset": "utf8mb4",
        "collation": "utf8mb4_unicode_ci",
        "autocommit": False,
    }

    def __new__(cls, config: Optional[Dict[str, Any]] = None):
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
            cls._instancia._config = config or cls.CONFIG_PADRAO
            cls._instancia._conn = None
        return cls._instancia

    def conectar(self):
        try:
            if self._conn is None or not self._conn.is_connected():
                self._conn = mysql.connector.connect(**self._config)
            return self._conn
        except MySQLError as e:
            raise ErroPersistencia(f"Falha ao conectar ao MySQL: {e}") from e

    def cursor(self, dictionary: bool = True):
        return self.conectar().cursor(dictionary=dictionary)

    def commit(self):
        self.conectar().commit()

    def rollback(self):
        if self._conn and self._conn.is_connected():
            self._conn.rollback()

    def fechar(self):
        if self._conn and self._conn.is_connected():
```

```
            self._conn.close()
            self._conn = None


class Tecnico:
    """Colaborador do laboratorio. Mapeia a tabela 'usuario'."""

    PAPEIS_VALIDOS = {"METROLOGISTA", "SIGNATARIO", "QUALIDADE"}

    def __init__(self, nome, email, senha_hash, papel,
                 registro_profissional=None, ativo=True, id_usuario=None):
        self.id_usuario = id_usuario
        self.nome = nome
        self.email = email
        self.senha_hash = senha_hash
        self.papel = papel
        self.registro_profissional = registro_profissional
        self.ativo = ativo
        self._validar()

    def _validar(self):
        if not self.nome or len(self.nome.strip()) < 3:
            raise ErroValidacao("Nome do tecnico deve ter ao menos 3 caracteres.")
        padrao_email = r"^[\w\.\-]+@[\w\.\-]+\.\w{2,}$"
        if not re.match(padrao_email, self.email or ""):
            raise ErroValidacao(f"E-mail invalido: {self.email}")
        if self.papel not in self.PAPEIS_VALIDOS:
            raise ErroValidacao(f"Papel invalido: {self.papel}")
        if self.papel == "SIGNATARIO" and not self.registro_profissional:
            raise ErroValidacao("Signatario exige registro profissional (RN06).")

    def __repr__(self):
        return f"<Tecnico id={self.id_usuario} nome='{self.nome}' papel={self.papel}>"


class Equipamento:
    """Instrumento de medicao. Mapeia a tabela 'instrumento'."""

    def __init__(self, tag, descricao, tipo, fabricante, modelo,
                 numero_serie, resolucao_mm, data_calibracao_prevista,
                 id_cliente, id_instrumento=None):
        self.id_instrumento = id_instrumento
        self.tag = tag
        self.descricao = descricao
        self.tipo = tipo
        self.fabricante = fabricante
        self.modelo = modelo
        self.numero_serie = numero_serie
        self.resolucao_mm = Decimal(str(resolucao_mm))
        self.data_calibracao_prevista = data_calibracao_prevista
        self.id_cliente = id_cliente
        self._validar()

    def _validar(self):
        if not self.tag or len(self.tag.strip()) < 3:
            raise ErroValidacao("Tag do equipamento deve ter ao menos 3 caracteres.")
        if self.resolucao_mm <= 0:
            raise ErroValidacao("Resolucao deve ser maior que zero.")
        if not isinstance(self.data_calibracao_prevista, date):
            raise ErroValidacao("Data de calibracao prevista invalida.")
        if not self.id_cliente or self.id_cliente <= 0:
            raise ErroValidacao("Equipamento exige um cliente valido (FK).")

    def calibracao_vencida(self) -> bool:
        return self.data_calibracao_prevista < date.today()

    def dias_para_vencer(self) -> int:
        return (self.data_calibracao_prevista - date.today()).days

    def __repr__(self):
        return f"<Equipamento id={self.id_instrumento} tag='{self.tag}' tipo='{self.tipo}'>"
```

```
class Medicao:
    """Ponto de medicao dimensional. Mapeia a tabela 'medicao'."""

    def __init__(self, ponto_medicao, valor_nominal, valor_medido,
                 repetitividade, incerteza, tolerancia=Decimal("0.0100"),
                 id_medicao=None):
        self.id_medicao = id_medicao
        self.ponto_medicao = ponto_medicao
        self.valor_nominal = Decimal(str(valor_nominal))
        self.valor_medido = Decimal(str(valor_medido))
        self.repetitividade = Decimal(str(repetitividade))
        self.incerteza = Decimal(str(incerteza))
        self.tolerancia = Decimal(str(tolerancia))
        self.erro_encontrado = self._calcular_erro()
        self._validar()

    def _calcular_erro(self) -> Decimal:
        """RN05: erro = soma algebrica (medido - nominal)."""
        return (self.valor_medido - self.valor_nominal).quantize(Decimal("0.0001"))

    def _validar(self):
        if not self.ponto_medicao or len(self.ponto_medicao.strip()) < 2:
            raise ErroValidacao("Ponto de medicao deve ser identificado.")
        if self.valor_medido < 0:
            raise ErroValidacao("Valor medido nao pode ser negativo.")
        if self.repetitividade < 0:
            raise ErroValidacao("Repetitividade nao pode ser negativa.")
        if self.incerteza < 0:
            raise ErroValidacao("Incerteza nao pode ser negativa.")

    def dentro_da_tolerancia(self) -> bool:
        return abs(self.erro_encontrado) <= self.tolerancia

    def classificar(self) -> str:
        if not self.dentro_da_tolerancia():
            return "REJEITADO"
        if abs(self.erro_encontrado) > (self.tolerancia * Decimal("0.8")):
            return "APROVADO COM RESTRICAO"
        return "APROVADO"

    def __repr__(self):
        return (f"<Medicao id={self.id_medicao} ponto='{self.ponto_medicao}' "
                f"erro={self.erro_encontrado} status={self.classificar()}>")


class TecnicoDAO:
    """CRUD da entidade Tecnico (tabela usuario)."""

    def __init__(self, conexao=None):
        self.db = conexao or Conexao()

    def inserir(self, tecnico):
        sql = """INSERT INTO usuario
                 (nome, email, senha_hash, papel, registro_profissional, ativo)
                 VALUES (%s, %s, %s, %s, %s, %s)"""
        try:
            cur = self.db.cursor()
            cur.execute(sql, (tecnico.nome, tecnico.email, tecnico.senha_hash,
                              tecnico.papel, tecnico.registro_profissional,
                              1 if tecnico.ativo else 0))
            self.db.commit()
            tecnico.id_usuario = cur.lastrowid
            cur.close()
            return tecnico.id_usuario
        except MySQLError as e:
            self.db.rollback()
            if e.errno == 1062:
                raise ErroPersistencia(f"E-mail ja cadastrado: {tecnico.email}") from e
            raise ErroPersistencia(f"Erro ao inserir tecnico: {e.msg}") from e

    def buscar_por_id(self, id_usuario):
        sql = "SELECT * FROM usuario WHERE id_usuario = %s"
        try:
            cur = self.db.cursor()
```

```
            cur.execute(sql, (id_usuario,))
            row = cur.fetchone()
            cur.close()
            return self._row_para_objeto(row) if row else None
        except MySQLError as e:
            raise ErroPersistencia(f"Erro na consulta: {e.msg}") from e

    def listar(self, apenas_ativos=True):
        sql = "SELECT * FROM usuario"
        if apenas_ativos:
            sql += " WHERE ativo = 1"
        sql += " ORDER BY nome"
        try:
            cur = self.db.cursor()
            cur.execute(sql)
            rows = cur.fetchall()
            cur.close()
            return [self._row_para_objeto(r) for r in rows]
        except MySQLError as e:
            raise ErroPersistencia(f"Erro na listagem: {e.msg}") from e

    def atualizar(self, tecnico):
        if not tecnico.id_usuario:
            raise ErroValidacao("Tecnico sem ID nao pode ser atualizado.")
        sql = """UPDATE usuario SET nome=%s, email=%s, papel=%s,
                 registro_profissional=%s, ativo=%s WHERE id_usuario=%s"""
        try:
            cur = self.db.cursor()
            cur.execute(sql, (tecnico.nome, tecnico.email, tecnico.papel,
                              tecnico.registro_profissional,
                              1 if tecnico.ativo else 0, tecnico.id_usuario))
            self.db.commit()
            afetadas = cur.rowcount
            cur.close()
            return afetadas > 0
        except MySQLError as e:
            self.db.rollback()
            raise ErroPersistencia(f"Erro ao atualizar tecnico: {e.msg}") from e

    def excluir(self, id_usuario):
        sql = "DELETE FROM usuario WHERE id_usuario = %s"
        try:
            cur = self.db.cursor()
            cur.execute(sql, (id_usuario,))
            self.db.commit()
            afetadas = cur.rowcount
            cur.close()
            return afetadas > 0
        except MySQLError as e:
            self.db.rollback()
            if e.errno == 1451:
                raise ErroPersistencia(
                    "Tecnico possui certificados vinculados e nao pode ser excluido.") from e
            raise ErroPersistencia(f"Erro ao excluir tecnico: {e.msg}") from e

    @staticmethod
    def _row_para_objeto(row):
        return Tecnico(id_usuario=row["id_usuario"], nome=row["nome"],
                       email=row["email"], senha_hash=row["senha_hash"],
                       papel=row["papel"],
                       registro_profissional=row.get("registro_profissional"),
                       ativo=bool(row["ativo"]))


class EquipamentoDAO:
    """CRUD da entidade Equipamento (tabela instrumento)."""

    def __init__(self, conexao=None):
        self.db = conexao or Conexao()

    def inserir(self, equip):
        sql = """INSERT INTO instrumento
                 (tag, descricao, tipo, fabricante, modelo, numero_serie,
                  resolucao_mm, data_calibracao_prevista, id_cliente)
                 VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)"""
```

```
        try:
            cur = self.db.cursor()
            cur.execute(sql, (equip.tag, equip.descricao, equip.tipo,
                              equip.fabricante, equip.modelo, equip.numero_serie,
                              equip.resolucao_mm, equip.data_calibracao_prevista,
                              equip.id_cliente))
            self.db.commit()
            equip.id_instrumento = cur.lastrowid
            cur.close()
            return equip.id_instrumento
        except MySQLError as e:
            self.db.rollback()
            if e.errno == 1062:
                raise ErroPersistencia(f"Tag ja cadastrada: {equip.tag}") from e
            if e.errno == 1452:
                raise ErroPersistencia(f"Cliente inexistente (id={equip.id_cliente}).") from e
            raise ErroPersistencia(f"Erro ao inserir equipamento: {e.msg}") from e

    def buscar_por_id(self, id_instrumento):
        sql = "SELECT * FROM instrumento WHERE id_instrumento = %s"
        try:
            cur = self.db.cursor()
            cur.execute(sql, (id_instrumento,))
            row = cur.fetchone()
            cur.close()
            return self._row_para_objeto(row) if row else None
        except MySQLError as e:
            raise ErroPersistencia(f"Erro na consulta: {e.msg}") from e

    def listar(self):
        sql = "SELECT * FROM instrumento ORDER BY tag"
        try:
            cur = self.db.cursor()
            cur.execute(sql)
            rows = cur.fetchall()
            cur.close()
            return [self._row_para_objeto(r) for r in rows]
        except MySQLError as e:
            raise ErroPersistencia(f"Erro na listagem: {e.msg}") from e

    def listar_calibracoes_vencidas(self):
        sql = """SELECT * FROM instrumento
                 WHERE data_calibracao_prevista < CURDATE()
                 ORDER BY data_calibracao_prevista"""
        try:
            cur = self.db.cursor()
            cur.execute(sql)
            rows = cur.fetchall()
            cur.close()
            return [self._row_para_objeto(r) for r in rows]
        except MySQLError as e:
            raise ErroPersistencia(f"Erro na consulta: {e.msg}") from e

    def atualizar(self, equip):
        if not equip.id_instrumento:
            raise ErroValidacao("Equipamento sem ID nao pode ser atualizado.")
        sql = """UPDATE instrumento SET descricao=%s, tipo=%s, fabricante=%s,
                 modelo=%s, numero_serie=%s, resolucao_mm=%s,
                 data_calibracao_prevista=%s, id_cliente=%s
                 WHERE id_instrumento=%s"""
        try:
            cur = self.db.cursor()
            cur.execute(sql, (equip.descricao, equip.tipo, equip.fabricante,
                              equip.modelo, equip.numero_serie, equip.resolucao_mm,
                              equip.data_calibracao_prevista, equip.id_cliente,
                              equip.id_instrumento))
            self.db.commit()
            afetadas = cur.rowcount
            cur.close()
            return afetadas > 0
        except MySQLError as e:
            self.db.rollback()
            raise ErroPersistencia(f"Erro ao atualizar equipamento: {e.msg}") from e

    def excluir(self, id_instrumento):
        sql = "DELETE FROM instrumento WHERE id_instrumento = %s"
        try:
```

```
            cur = self.db.cursor()
            cur.execute(sql, (id_instrumento,))
            self.db.commit()
            afetadas = cur.rowcount
            cur.close()
            return afetadas > 0
        except MySQLError as e:
            self.db.rollback()
            if e.errno == 1451:
                raise ErroPersistencia(
                    "Equipamento possui certificados vinculados e nao pode ser excluido.")
from e
            raise ErroPersistencia(f"Erro ao excluir equipamento: {e.msg}") from e

    @staticmethod
    def _row_para_objeto(row):
        return Equipamento(id_instrumento=row["id_instrumento"], tag=row["tag"],
                           descricao=row["descricao"], tipo=row["tipo"],
                           fabricante=row["fabricante"], modelo=row["modelo"],
                           numero_serie=row["numero_serie"],
                           resolucao_mm=row["resolucao_mm"],
                           data_calibracao_prevista=row["data_calibracao_prevista"],
                           id_cliente=row["id_cliente"])


class MedicaoDAO:
    """CRUD da entidade Medicao (tabela medicao)."""

    def __init__(self, conexao=None):
        self.db = conexao or Conexao()

    def inserir(self, medicao):
        sql = """INSERT INTO medicao
                 (ponto_medicao, valor_nominal, valor_medido,
                  erro_encontrado, repetitividade, incerteza)
                 VALUES (%s, %s, %s, %s, %s, %s)"""
        try:
            cur = self.db.cursor()
            cur.execute(sql, (medicao.ponto_medicao, medicao.valor_nominal,
                              medicao.valor_medido, medicao.erro_encontrado,
                              medicao.repetitividade, medicao.incerteza))
            self.db.commit()
            medicao.id_medicao = cur.lastrowid
            cur.close()
            return medicao.id_medicao
        except MySQLError as e:
            self.db.rollback()
            raise ErroPersistencia(f"Erro ao inserir medicao: {e.msg}") from e

    def buscar_por_id(self, id_medicao):
        sql = "SELECT * FROM medicao WHERE id_medicao = %s"
        try:
            cur = self.db.cursor()
            cur.execute(sql, (id_medicao,))
            row = cur.fetchone()
            cur.close()
            return self. row para objeto(row) if row else None
        except MySQLError as e:
            raise ErroPersistencia(f"Erro na consulta: {e.msg}") from e

    def listar(self):
        sql = "SELECT * FROM medicao ORDER BY id_medicao"
        try:
            cur = self.db.cursor()
            cur.execute(sql)
            rows = cur.fetchall()
            cur.close()
            return [self._row_para_objeto(r) for r in rows]
        except MySQLError as e:
            raise ErroPersistencia(f"Erro na listagem: {e.msg}") from e

    def listar_por_certificado(self, id_certificado):
        sql = """SELECT m.* FROM medicao m
                 INNER JOIN certificado_medicao cm ON cm.id_medicao = m.id_medicao
                 WHERE cm.id_certificado = %s
```

```
                ORDER BY cm.ordem_apresentacao"""
        try:
            cur = self.db.cursor()
            cur.execute(sql, (id_certificado,))
            rows = cur.fetchall()
            cur.close()
            return [self._row_para_objeto(r) for r in rows]
        except MySQLError as e:
            raise ErroPersistencia(f"Erro na consulta: {e.msg}") from e

    def atualizar(self, medicao):
        if not medicao.id_medicao:
            raise ErroValidacao("Medicao sem ID nao pode ser atualizada.")
        sql = """UPDATE medicao SET ponto_medicao=%s, valor_nominal=%s,
                 valor_medido=%s, erro_encontrado=%s, repetitividade=%s,
                 incerteza=%s WHERE id_medicao=%s"""
        try:
            cur = self.db.cursor()
            cur.execute(sql, (medicao.ponto_medicao, medicao.valor_nominal,
                              medicao.valor_medido, medicao.erro_encontrado,
                              medicao.repetitividade, medicao.incerteza,
                              medicao.id_medicao))
            self.db.commit()
            afetadas = cur.rowcount
            cur.close()
            return afetadas > 0
        except MySQLError as e:
            self.db.rollback()
            raise ErroPersistencia(f"Erro ao atualizar medicao: {e.msg}") from e

    def excluir(self, id_medicao):
        sql = "DELETE FROM medicao WHERE id_medicao = %s"
        try:
            cur = self.db.cursor()
            cur.execute(sql, (id_medicao,))
            self.db.commit()
            afetadas = cur.rowcount
            cur.close()
            return afetadas > 0
        except MySQLError as e:
            self.db.rollback()
            if e.errno == 1451:
                raise ErroPersistencia(
                    "Medicao esta associada a certificados e nao pode ser excluida.") from e
            raise ErroPersistencia(f"Erro ao excluir medicao: {e.msg}") from e

    @staticmethod
    def _row_para_objeto(row):
        return Medicao(id_medicao=row["id_medicao"],
                       ponto_medicao=row["ponto_medicao"],
                       valor_nominal=row["valor_nominal"],
                       valor_medido=row["valor_medido"],
                       repetitividade=row["repetitividade"],
                       incerteza=row["incerteza"])
```

APÊNDICE B — CÓDIGO-FONTE: 05_main.py

```
"""
PROJETO APLICADO I - UniSENAI Florianopolis
AV03 - MVP Parte 1: Classe Principal e Demonstracao de Fluxo
Arquivo: 05_main.py
"""

import sys
from datetime import date, timedelta
from decimal import Decimal

from mysql.connector import Error as MySQLError

from classes import (
    Conexao, Tecnico, Equipamento, Medicao,
    TecnicoDAO, EquipamentoDAO, MedicaoDAO,
    ErroValidacao, ErroPersistencia,
)

LARGURA = 78



def titulo(texto):
    print("\n" + "=" * LARGURA)
    print(f"  {texto}")
    print("=" * LARGURA)



def subtitulo(texto):
    print(f"\n--- {texto} " + "-" * (LARGURA - len(texto) - 5))



def linha(campos):
    for chave, valor in campos.items():
        print(f"  {chave:<24} {valor}")



class SistemaCalibracao:
    """Classe orquestradora do MVP."""

    def __init__(self):
        self.db = Conexao()
        self.tecnicos = TecnicoDAO(self.db)
        self.equipamentos = EquipamentoDAO(self.db)
        self.medicoes = MedicaoDAO(self.db)

    def testar_conexao(self):
        titulo("1. TESTE DE CONEXAO COM O BANCO DE DADOS")
        try:
            cur = self.db.cursor()
            cur.execute("SELECT VERSION(), DATABASE(), NOW()")
            row = cur.fetchone()
            cur.close()
            linha({"Servidor MySQL": row["VERSION()"],
                   "Banco de dados": row["DATABASE()"],
                   "Data/hora servidor": row["NOW()"]})
            print("\n  [OK] Conexao estabelecida com sucesso.")
            return True
        except ErroPersistencia as e:
            print(f"\n  [FALHA] {e}")
            print("\n  Verifique se o MySQL esta rodando e se o banco")
            print("  'bd_certificados_schulz' foi criado (script da AV02).")
            return False

    def demonstrar_crud(self):
        titulo("2. DEMONSTRACAO DE OPERACOES CRUD")

        subtitulo("2.1 INSERT - Novo tecnico metrologista")
        try:
            novo_tecnico = Tecnico(
```

```
            nome="Ana Paula Ferreira",
            email="ana.ferreira@schulz.com.br",
            senha_hash="$2y$12$demoHashAV03Metrologista",
            papel="METROLOGISTA",
            registro_profissional="TEC-MET-9901/SC")
        id_tec = self.tecnicos.inserir(novo_tecnico)
        linha({"ID gerado": id_tec, "Nome": novo_tecnico.nome,
               "Papel": novo_tecnico.papel, "Status": "Inserido com sucesso"})
    except (ErroValidacao, ErroPersistencia) as e:
        print(f"  [ERRO] {e}")

    subtitulo("2.2 INSERT - Novo equipamento")
    try:
        novo_equip = Equipamento(
            tag="MIC-EXT-099",
            descricao="Micrometro Externo Digital 0-25mm (novo lote)",
            tipo="Micrometro", fabricante="Mitutoyo",
            modelo="293-240-30", numero_serie="MT-9911001",
            resolucao_mm=Decimal("0.00100"),
            data_calibracao_prevista=date.today() + timedelta(days=365),
            id_cliente=1)
        id_eq = self.equipamentos.inserir(novo_equip)
        linha({"ID gerado": id_eq, "Tag": novo_equip.tag,
               "Cliente (FK)": novo_equip.id_cliente,
               "Status": "Inserido com sucesso"})
    except (ErroValidacao, ErroPersistencia) as e:
        print(f"  [ERRO] {e}")

    subtitulo("2.3 INSERT - Nova medicao com validacao de tolerancia")
    try:
        nova_med = Medicao(
            ponto_medicao="P1 - Diametro Externo (AV03)",
            valor_nominal=Decimal("12.0000"),
            valor_medido=Decimal("12.0035"),
            repetitividade=Decimal("0.00015"),
            incerteza=Decimal("0.00120"),
            tolerancia=Decimal("0.0100"))
        id_med = self.medicoes.inserir(nova_med)
        linha({"ID gerado": id_med, "Ponto": nova_med.ponto_medicao,
               "Erro calculado": f"{nova_med.erro_encontrado} mm",
               "Tolerancia": f"+-{nova_med.tolerancia} mm",
               "Parecer": nova_med.classificar()})
    except (ErroValidacao, ErroPersistencia) as e:
        print(f"  [ERRO] {e}")

    subtitulo("2.4 SELECT - Listagem de tecnicos ativos")
    try:
        lista = self.tecnicos.listar(apenas_ativos=True)
        print(f"  Total encontrado: {len(lista)}\n")
        print(f"  {'ID':<5} {'NOME':<32} {'PAPEL':<14} {'REGISTRO':<16}")
        print(f"  {'-'*5} {'-'*32} {'-'*14} {'-'*16}")
        for t in lista[:8]:
            print(f"  {t.id_usuario:<5} {t.nome[:32]:<32} "
                  f"{t.papel:<14} {(t.registro_profissional or '-')[:16]:<16}")
    except ErroPersistencia as e:
        print(f"  [ERRO] {e}")

    subtitulo("2.5 UPDATE - Alteracao de dados")
    try:
        if 'id_tec' in locals():
            tecnico = self.tecnicos.buscar_por_id(id_tec)
            if tecnico:
                tecnico.nome = "Ana Paula Ferreira Lima"
                ok = self.tecnicos.atualizar(tecnico)
                linha({"ID": tecnico.id_usuario, "Novo nome": tecnico.nome,
                       "Status": "Atualizado" if ok else "Nao encontrado"})
    except (ErroValidacao, ErroPersistencia) as e:
        print(f"  [ERRO] {e}")

    subtitulo("2.6 DELETE - Remocao controlada")
    try:
        if 'id_med' in locals():
            ok = self.medicoes.excluir(id_med)
            linha({"ID removido": id_med,
                   "Status": "Excluido" if ok else "Nao encontrado"})
    except ErroPersistencia as e:
        print(f"  [ERRO] {e}")
```

```
    def demonstrar_validacoes(self):
        titulo("3. VALIDACAO DE REGRAS DE NEGOCIO")

        subtitulo("3.1 RN06 - Signatario sem registro profissional")
        try:
            Tecnico(nome="Teste Invalido", email="teste@schulz.com.br",
                    senha_hash="hash", papel="SIGNATARIO",
                    registro_profissional=None)
            print("  [FALHA] Deveria ter lancado excecao.")
        except ErroValidacao as e:
            print(f"  [OK] Excecao capturada: {e}")

        subtitulo("3.2 E-mail com formato invalido")
        try:
            Tecnico(nome="Teste Email", email="email-sem-arroba",
                    senha_hash="hash", papel="METROLOGISTA")
            print("  [FALHA] Deveria ter lancado excecao.")
        except ErroValidacao as e:
            print(f"  [OK] Excecao capturada: {e}")

        subtitulo("3.3 RN05 - Calculo automatico do erro (soma algebrica)")
        m1 = Medicao("Teste A", Decimal("10.0000"), Decimal("10.0003"),
                     Decimal("0.00008"), Decimal("0.00045"))
        m2 = Medicao("Teste B", Decimal("10.0000"), Decimal("9.9995"),
                     Decimal("0.00008"), Decimal("0.00045"))
        print(f"  Medicao A: nominal=10.0000 medido=10.0003 -> erro={m1.erro_encontrado}
({m1.classificar()})")
        print(f"  Medicao B: nominal=10.0000 medido=9.9995  -> erro={m2.erro_encontrado}
({m2.classificar()})")

        subtitulo("3.4 Validacao de tolerancia - ponto fora do limite")
        m3 = Medicao("Teste C", Decimal("20.0000"), Decimal("20.0250"),
                     Decimal("0.00200"), Decimal("0.01200"),
                     tolerancia=Decimal("0.0100"))
        print(f"  Erro={m3.erro_encontrado} | Tolerancia=+-{m3.tolerancia} |
Parecer={m3.classificar()}")
        if not m3.dentro_da_tolerancia():
            print("  [OK] Ponto corretamente reprovado pela tolerancia.")

    def relatorio_certificados(self):
        titulo("4. RELATORIO 1 - CERTIFICADOS COM INSTRUMENTO E MEDICOES")
        sql = """SELECT c.numero, c.status, c.resultado, c.data_emissao,
                        i.tag, i.tipo, cl.razao_social,
                        COUNT(cm.id_medicao) AS total_medicoes
                 FROM certificado c
                 INNER JOIN instrumento i ON i.id_instrumento = c.id_instrumento
                 INNER JOIN cliente cl ON cl.id_cliente = i.id_cliente
                 LEFT JOIN certificado_medicao cm ON cm.id_certificado = c.id_certificado
                 GROUP BY c.id_certificado, c.numero, c.status, c.resultado,
                          c.data_emissao, i.tag, i.tipo, cl.razao_social
                 ORDER BY c.data_emissao DESC"""
        try:
            cur = self.db.cursor()
            cur.execute(sql)
            rows = cur.fetchall()
            cur.close()
            print(f"  Total de certificados: {len(rows)}\n")
            print(f"  {'NUMERO':<12} {'TAG':<14} {'STATUS':<20} {'RESULTADO':<24} {'MED':<4}")
            print(f"  {'-'*12} {'-'*14} {'-'*20} {'-'*24} {'-'*4}")
            for r in rows:
                print(f"  {r['numero']:<12} {r['tag']:<14} {r['status']:<20} "
                      f"{r['resultado']:<24} {r['total_medicoes']:<4}")
        except MySQLError as e:
            print(f"  [ERRO] {e.msg}")

    def relatorio_pendentes(self):
        titulo("5. RELATORIO 2 - CERTIFICADOS PENDENTES E APROVADOS")
        sql = """SELECT c.numero, c.status, c.resultado,
                        u.nome AS metrologista, s.nome AS signatario
                 FROM certificado c
                 INNER JOIN usuario u ON u.id_usuario = c.id_metrologista
                 LEFT JOIN usuario s ON s.id_usuario = c.id_signatario
                 WHERE c.status IN ('PENDENTE_APROVACAO', 'APROVADO')
                 ORDER BY c.status, c.numero"""
        try:
```

```
            cur = self.db.cursor()
            cur.execute(sql)
            rows = cur.fetchall()
            cur.close()
            pendentes = [r for r in rows if r["status"] == "PENDENTE_APROVACAO"]
            aprovados = [r for r in rows if r["status"] == "APROVADO"]
            print(f"  Pendentes de aprovacao: {len(pendentes)}")
            print(f"  Aprovados: {len(aprovados)}\n")
            print(f"  {'NUMERO':<12} {'STATUS':<22} {'METROLOGISTA':<28} {'SIGNATARIO':<24}")
            print(f"  {'-'*12} {'-'*22} {'-'*28} {'-'*24}")
            for r in rows:
                print(f"  {r['numero']:<12} {r['status']:<22} "
                      f"{r['metrologista'][:28]:<28} "
                      f"{(r['signatario'] or '(pendente)')[:24]:<24}")
        except MySQLError as e:
            print(f"  [ERRO] {e.msg}")

    def relatorio_rastreabilidade(self):
        titulo("6. RELATORIO 3 - RASTREABILIDADE: MEDICOES E PADROES")
        sql = """SELECT m.ponto_medicao, m.valor_nominal, m.valor_medido,
                        m.erro_encontrado, p.codigo_identificacao AS padrao,
                        p.certificado_rastreabilidade AS laudo_rbc,
                        p.validade AS validade_padrao, mp.lote_utilizacao
                 FROM medicao m
                 INNER JOIN medicao_padrao mp ON mp.id_medicao = m.id_medicao
                 INNER JOIN padrao_calibracao p ON p.id_padrao = mp.id_padrao
                 ORDER BY m.id_medicao, p.codigo_identificacao"""
        try:
            cur = self.db.cursor()
            cur.execute(sql)
            rows = cur.fetchall()
            cur.close()
            print(f"  Total de vinculos medicao-padrao: {len(rows)}\n")
            print(f"  {'PONTO DE MEDICAO':<34} {'PADRAO':<14} {'ERRO':<12} {'VALIDADE':<12}")
            print(f"  {'-'*34} {'-'*14} {'-'*12} {'-'*12}")
            for r in rows:
                print(f"  {r['ponto_medicao'][:34]:<34} {r['padrao']:<14} "
                      f"{str(r['erro_encontrado']):<12} {str(r['validade_padrao']):<12}")
        except MySQLError as e:
            print(f"  [ERRO] {e.msg}")

    def relatorio_alertas(self):
        titulo("7. ALERTAS - CALIBRACOES VENCIDAS (RN03)")
        try:
            vencidos = self.equipamentos.listar_calibracoes_vencidas()
            if not vencidos:
                print("  Nenhum equipamento com calibracao vencida.")
                return
            print(f"  Total de equipamentos vencidos: {len(vencidos)}\n")
            for e in vencidos:
                print(f"  {e.tag:<16} {e.tipo:<16} venceu em "
                      f"{e.data_calibracao_prevista} ({abs(e.dias_para_vencer())} dias)")
        except ErroPersistencia as e:
            print(f"  [ERRO] {e}")

    def executar(self):
        print("\n" + "#" * LARGURA)
        print("#" + " " * (LARGURA - 2) + "#")
        print("#" + "  SISTEMA DE CERTIFICADOS DE CALIBRACAO - SCHULZ S.A.".center(LARGURA -
2) + "#")
        print("#" + "  AV03 - MVP Parte 1 | POO + Persistencia MySQL".center(LARGURA - 2) +
"#")
        print("#" + " " * (LARGURA - 2) + "#")
        print("#" * LARGURA)
        if not self.testar_conexao():
            print("\n  Execucao interrompida.")
            return
        self.demonstrar_crud()
        self.demonstrar_validacoes()
        self.relatorio_certificados()
        self.relatorio_pendentes()
        self.relatorio_rastreabilidade()
        self.relatorio_alertas()
        titulo("EXECUCAO CONCLUIDA COM SUCESSO")
        print("  Todos os modulos foram executados sem erros fatais.\n")

    def encerrar(self):
```

```
        self.db.fechar()


def main():
    sistema = None
    try:
        sistema = SistemaCalibracao()
        sistema.executar()
    except KeyboardInterrupt:
        print("\n\n  Execucao interrompida pelo usuario.")
    except ErroPersistencia as e:
        print(f"\n  [ERRO DE PERSISTENCIA] {e}")
    except Exception as e:
        print(f"\n  [ERRO INESPERADO] {type(e).__name__}: {e}")
    finally:
        if sistema:
            sistema.encerrar()
        sys.exit(0)


if __name__ == "__main__":
    main()
```

APÊNDICE C — INSTRUÇÕES DE EXECUÇÃO

Para executar a solução da AV03 em um ambiente local, siga os passos abaixo na ordem indicada:

8. **Passo 1 —** Instalar a dependência do driver de conexão: executar no terminal o comando pip install mysql-connector-python.
9. **Passo 2 —** Executar o script de setup do banco de dados (setup_av03.sql) no MySQL Workbench, criando o schema e as tabelas do modelo da AV02.
10. **Passo 3 —** Ajustar host, usuário e senha no dicionário CONFIG_PADRAO da classe Conexao, conforme as credenciais do ambiente MySQL local.
11. **Passo 4 —** Executar o programa principal com o comando python 05_main.py.
12. **Passo 5 —** Conferir a saída dos sete blocos no terminal: conexão, CRUD, validações, relatórios de certificados, pendentes/aprovados, rastreabilidade e alertas de calibrações vencidas.